Crucible

An adversarial security platform that attacks an AI system, verifies its work with checks the system can never see, hardens it in a closed loop, and then measures its own catch rate against an adversary that already knows the scheme.

Direction: ML + LLM hybrid • **Showcase:** June 29, 2026, 10-minute live demo • **Team:** 3 to 4 (clean four-way split, see section 7)

> **Summary.** Crucible is a target-agnostic red-team and blue-team platform. An LLM-driven adversary discovers semantically valid ways to defeat a target system: evading a fraud model, reward-hacking a coding agent, or violating the spec of a research agent. An ensemble of independent oracles, generated from a sealed spec and unable to collude, catches those attacks without trusting the thing being checked. A blue loop hardens the target automatically. And the platform continuously red-teams itself to report, as a single number, how often a cheat still gets through.

1. The problem, and why it is technically hard

As we hand more work to AI systems, the bottleneck stops being production and becomes trust. The same failure shows up in two places that look different but are one problem. A fraud detector has blind spots: adversaries adapt and probe for behavior that slips past it, and those blind spots are found today by manual red-teaming and post-mortem review, which does not scale and always lags the attacker. An AI agent reward-hacks: it satisfies the letter of a task while violating its intent, in ways a quick human pass misses. In both cases the model looks strong on its own distribution and fails exactly where an adaptive adversary pushes.

The naive automated fixes break on the data these systems actually live in. Classic gradient-based adversarial attacks produce samples that fool a tabular or time-series fraud model mathematically but are illegal or meaningless in the real world: negative transaction amounts, impossible timestamps, violated business rules. They tell you nothing actionable. For agent output the mirror failure holds: any fixed test suite is something the producer can learn to pass without doing the real work, a large language model judge shares the producer's blind spots, and a human cannot review everything.

The unsolved core is **independent verification**: checks that establish ground truth without trusting the thing being checked, and that still hold when the producer knows exactly how it is being checked, coupled with a search that finds the blind spots in the first place and a loop that feeds them back automatically. This is not hyperparameter tuning. It is a compound problem of adversarial search, constraint satisfaction, non-colluding verification, and closed-loop optimization, against an adversary that reasons about why it was caught.

2. The idea: one platform, four moving parts

Crucible is not a verifier and it is not just a fuzzer. It is a self-measuring adversarial security platform built around a single loop.

and re-attack — the loop closes on itself and grades each pass

It is **target agnostic**. The same core runs over a fraud model, a code-producing agent, and a multi-step research agent through a thin adapter; the target is simply the system under test, and oracle access to it is a first-class tool inside the loop rather than an afterthought. AI is a primitive on two sides, as the adversarial search engine and as the hardening engine, not a bolt-on. Using an LLM's semantic reasoning in place of gradients as the search engine

is a genuinely different design philosophy from standard adversarial-ML pipelines.

3. Architecture: every piece, and who owns it

Four pillars carry the platform. Each is a clean ownership boundary for one team member, and each folds in the distinct features of both source designs.

Pillar 1 — Targets and independent oracles

A thin adapter wraps each system under test, so the core never changes when the target does.

- **Target adapters.** A LightGBM fraud detector on a public dataset (IEEE-CIS Fraud Detection or the Kaggle credit-card set), swappable for an autoencoder, GMM, or DAGMM unsupervised time-series anomaly detector where the deeper research bet lives; a code-producing agent; and a multi-step research agent. One core, three adapters.
- **Spec sealing and a producer sandbox.** The target runs with no path to the verification artifacts. This independence is the foundation everything else rests on.
- **Four independent oracles,** derived from the sealed spec and never from the submission, each failing differently so a hack that slips past one is caught by another: held-out tests generated after submission and never exposed; metamorphic relations (if the input changes this way, the output must change that way) that need no reference answer; a differential oracle that spawns a second implementation from a different model family and requires agreement; and property-based fuzzing on invariants. A large language model judge is included as one measured approach among several, never the sole authority.

Pillar 2 — Red: the adversarial search engine

- **LLM as search engine.** The red agent receives a flagged case or a produced artifact, reasons about why it was caught, proposes the minimal modification that preserves intent (fraudulent intent, or a passing-but-cheating answer) while satisfying domain constraints, queries the target for a score through `query_target()`, and iterates toward evasion.
- **Strategy catalog.** Successful tactics are distilled into persistent memory the agent reuses and builds on across runs (MCP-style memory and observability).
- **White-box adversary.** A second mode hands the agent the full verification strategy and rewards it for finding a cheap cheat anyway. This converts the claim ‘we catch cheats’ into a measured recall number.
- **Hybrid fallback.** When constraint satisfaction is hard, the LLM proposes the strategy and a small constrained numerical search executes it, so the system degrades gracefully instead of failing.

Pillar 3 — Blue: automated hardening

A second LLM reads the strategy catalog and automatically proposes new features, generates adversarial training samples, or recommends a specialist ensemble; the target is retrained and re-evaluated. This is the rapid-response patch: once an attack is found, close the hole and verify it is closed on held-out attacks, not on the attacks used to build the patch.

Pillar 4 — Measure: self-measurement, observability, and benchmark

- **Honest dashboard.** Undetected-hack rate; the gap between validation pass-rate and held-out pass-rate (the headline metric); verifier recall on the seeded corpus; cost per caught hack; and human minutes per thousand units of output.
- **Reasoning and audit traces.** Every agent step is traced. Every verdict ships an audit trace naming the exact check that fired, the spec obligation it violated, and a one-click replay.
- **Curves.** Attack-success-rate and detection-rate plotted across rounds; red and blue co-evolution over N rounds producing a convergence or oscillation curve.
- **Artifacts.** A seeded hack corpus and leaderboard exported as a reusable benchmark that outlives the demo, plus an auto-drafted SR 11-7 style model risk report.

4. The core bet

The single hardest piece, and the reason the project is worth attempting, is **independent oracle generation that survives a white-box adversary**. If the oracle holds when the producer knows the scheme, and if LLM-driven semantic adversarial search produces valid, interpretable evasions on tabular and time-series data that beat or complement random and gradient search, we have a result a frontier lab would cite. If we instead ship a polished dashboard over a gameable check, we have a circular argument dressed up as a product. We spend our scarce two weeks on the hard half first and treat the interface as the thing we build once the oracle is real.

5. Why it resonates with frontier-lab safety priorities

Component for component, Crucible is a working testbed for problems on Anthropic's own published list of technical safety research directions. The point is not that a two-week capstone solves them; it is that the capstone produces a measured, reusable artifact pointed straight at the questions a safety lab is asking.

Crucible mechanism	Named open problem it instantiates
Validation vs held-out pass-rate gap	Scalable oversight under **systematic, exploitable oversight error**: the gap is exactly the proxy reward R minus the ground truth R*, which a producer can learn to exploit.
Non-colluding oracle ensemble over a sealed spec	**AI control**: reducing actor and monitor collusion; a trusted screen the actor cannot subvert or see.
Differential cross-family oracle	**Anomaly detection between two models**: flag inputs where a second implementation from a different family disagrees.
Blue loop / rapid-response patch	**Adaptive defenses** that respond to a discovered attack rather than static pre-deployment hardening.
Exported seeded-hack corpus and leaderboard	**Realistic, differential benchmarks** for adversarial robustness that outlive the demo.
Audit trace and reasoning trace per verdict	Auditable, faithful **externalized reasoning** for after-the-fact review.
Halt certification at a residual red line	Capability-threshold gating in the spirit of a **Responsible Scaling Policy**: stop certifying when the residual crosses a pre-set bound.

6. Scope: what ships in two weeks

Core, committed (the honest minimum):

- Minimal spec sealing and a producer sandbox with no path to the verification artifacts.
- The four independent oracles running in parallel over one real domain (code), plus the fraud-model adapter so the red and blue loop demos visually.
- The red agent finding at least three distinct evasion or reward-hack strategies with a measurable attack-success-rate, and the white-box adversarial self-test measuring recall live.
- One blue-hardening round showing detection rate recover.
- The measurement dashboard, audit traces, and an export of the seeded corpus and leaderboard as a reusable benchmark. This version alone is already a strong showing.

Stretch, if the core lands early:

- A time-series anomaly-detection target (the research-grade path).
- Red and blue co-evolution over N rounds with a convergence or oscillation curve.
- An automated spec compiler that turns free-form tasks into machine-checkable obligations.
- The full verifier tournament: a learned router that picks the cheapest verifier per task type, hierarchical decomposition, debate escalation to a human tiebreak, and a live human-review-budget meter.

- A polished interactive dashboard and the auto-generated model risk report.

7. Team: four ownable pieces

The workflow splits cleanly into four pieces, so anyone joining sees immediately what they would own.

- **Targets and oracles**: adapters, spec sealing and sandbox, the four-oracle verification ensemble (tabular and time-series anomaly detection included).
- **Red agent**: LLM-driven adversarial search, strategy catalog, white-box adversary, hybrid fallback.
- **Blue loop**: automated hardening and retraining closed loop.
- **Measure**: traces, attack-success and detection curves, co-evolution curve, dashboard, exported benchmark, and model risk report.

The pitch: we build an AI that breaks a system and then heals it, red team and blue team both LLM-driven, and the whole rig grades its own catch rate against an adversary that already knows the scheme. It lands on four hot themes at once: agents, adversarial robustness, scalable oversight, and AI safety.

8. Risks and honest limits: the technical bet, stated plainly

We do not pretend the deep limits are solved. For each, Crucible shrinks the gap with a concrete mechanism, reports the residual as a number, and halts certification when that residual crosses a red line. Measuring the limits honestly is the artifact, not hiding them.

- **The LLM fails to produce valid adversarial samples** (constraint satisfaction fails): degrade to the hybrid, where the LLM proposes the strategy and a small constrained search executes it numerically.
- **The blue loop overfits to known attacks or does not converge**: measure generalization on a held-out attack set defined up front.
- **The oracles share blind spots**: four mechanisms that fail differently, and the white-box adversary is rewarded precisely for finding a shared gap.
- **The spec is a proxy for intent, unknown hack classes exist, and oversight erodes as the producer outpaces the verifier**: report verifier recall on the seeded corpus continuously and stop certifying when it drops below the red line.
- **Per-loop LLM cost and latency**: cache, batch, and run a small model on the inner loop.
- **Self-deceiving evaluation**: freeze the evaluation protocol on day one.

9. The demo: June 29, 2026, 10-minute live sketch

0–2 min	The problem: detectors and agents both have blind spots, and manual review does not scale.
2–6 min	Point the red agent at the fraud detector live. Watch it discover an evasion in real time, attack-success-rate climbing, with the agent's reasoning trace scrolling alongside. Then point the same red agent at a code task and watch it reward-hack a fixed test suite while the independent oracles catch it.
6–9 min	Trigger the blue hardening, retrain, and show detection rate recover live. The same agent now gets blocked.
9–10 min	Show the self-measurement dashboard (undetected-hack rate and the validation vs held-out gap), the strategy catalog, the exported benchmark, and the auto-generated model risk report.

Safety-direction references: Anthropic Alignment Science, Recommendations for Technical AI Safety Research Directions (alignment.anthropic.com/2025/recommended-directions); Responsible Scaling Policy (anthropic.com/responsible-scaling-policy).